



Hiding Traffic Fingerprints

Final Report

Technical Director

William Layton

william.layton@cyber.nsa.gov

University Project Supervisors

Bhavani Thuraisingham

bxt043000@utdallas.edu

Brian Ricks

bwr031000@utdallas.edu

Team 1

Ruiyu Mao

rxm210041@utdallas.edu

Joseph Norman

jsn180000@utdallas.edu

Tram Pham

txp170008@utdallas.edu

Marlon Schumacher

mws180000@utdallas.edu

Alinta Wang

ajw170130@utdallas.edu

April 1, 2021

Keywords: Network traffic, traffic fingerprinting, adversarial models, machine learning, computer networks, privacy, encryption, metadata, deep learning, classifiers, neural network

Table of Contents

Project Summary	3
Description	3
Motivation	3
Team Responsibilities	3
Budget	3
Deliverables	4
Broader Impact	4
Plan	4
Tentative Schedule	4
Recurring Meeting Schedule	4
Final Review	5
Project Approach	5
Project Timeline	5
Final Progress	5
Experimentation	6
Training the Deep Learning Web Fingerprinting Attack	8
Weka Explorer	8
Challenges	11
Conclusion	12
Future Plans	12
Literature Review	12
References	16
Investigator Biographies	17

Project Summary

Description

In recent years, the rise of deep learning has been exponentially expanding and developing in today's society. Today, thousands of homes own their personal smart speakers and are vulnerable to privacy leakage over traffic attacks. Within today's modern computer systems, information can leak through software-based side channels, such as the CPU cache or the DRAM [2]. An adversary can intercept incoming and outgoing encrypted traffic - revealing users' activities and sensitive information, which underscores the gravity of building an automated defense system. An adversary can use several methods to infer user input - primarily utilizing the side channel to obtain keystroke timings and monitor cache lines [1]. Thus, it is vital to research and understand the conceptual attacks and crucial to discover what other methods adversaries can use to attack and to what extent leveraging deep learning and experiment over privacy leakage over encrypted traffic.

Motivation

As the Internet continues to become an essential part of daily life, people provide increasing amounts of personal information to various online services and communicate with each other online more frequently. This has led to a rapidly growing concern of privacy violations since sensitive information can be leaked when transmitted over a network. While encrypting network connections greatly improves the security of transmitted information, attackers can work their way around currently used encryption algorithms to continue fingerprinting traffic, but to what extent is unclear. Contributing to the ongoing research in this area will help establish a better understanding of how dangerous traffic fingerprinting is and how to defend against it.

Team Responsibilities

- ❖ Conduct research and gain a thorough understanding of topic
- ❖ Examine and demonstrate traffic fingerprinting
- ❖ Examine and derive results
- ❖ Attend meetings and contribute to reports

Budget

We do not expect any financial costs to this project. We expect to collectively work 8-10 hours per week on this project and meet at least twice per week.

Deliverables

- ❖ Proposal and Literature Review
- ❖ Midterm Presentation
- ❖ Final Presentation

Broader Impact

As there are many research papers and publications with different findings and approaches to addressing the issue of hiding traffic fingerprints, there is currently a lot of unclear information on the issue of network traffic fingerprints and how to hide them. By conducting this research project, any findings or discoveries made will improve the world's understanding of how dangerous traffic fingerprinting is, the different ways attackers can observe the contents of network traffic, and the best ways to defend against such attacks.

Plan

Meetings will be scheduled to meet and be consulted by the sponsor on every Monday of the week.

Tentative Schedule

February 4, 2022	Submit Proposal
February 11, 2022	Research and explore topics
February 25, 2022	Set up environment and experiments
March 31, 2022	Submit Midterm Report
April 8, 2022	Continue researching more topics and set up more experiments
May 5, 2022	Final Report

Recurring Meeting Schedule

Monday	12:00 P.M. - 12:30 P.M. CST/CDT
Friday	1:00 P.M. - 3:45 P.M. CST/CDT

Final Review

Project Approach

While encrypting network connections greatly improves the security of transmitting information, attackers are still able to determine a user's web activity through various techniques within website fingerprinting. Some example features include packet length, moderating the interval between packets, upstream bandwidth, and downstream bandwidth. Our team seeks to understand this attack and observe the best ways to defend against it. The goals for our project include gaining a thorough understanding and quantification of the threat of traffic fingerprinting and how much we need to do to protect against this attack. We also hope to provide insight into ways traffic fingerprinting can be done and find potential ways to mitigate traffic fingerprinting. The team initiated research through papers and exploration, and examined models presented. The team then set up an environment to simulate these models alongside our variations to develop a better understanding. We then built and expanded upon the models through simulation and testing variations. Finally, we conduct experiments through the models to analyze the results and further investigate other solutions.

Project Timeline

February 4 - Submit Proposal

February 11 - Research and explore topics

February 25 - Setup environment and experiment

April 1 - Midterm Report and Presentation

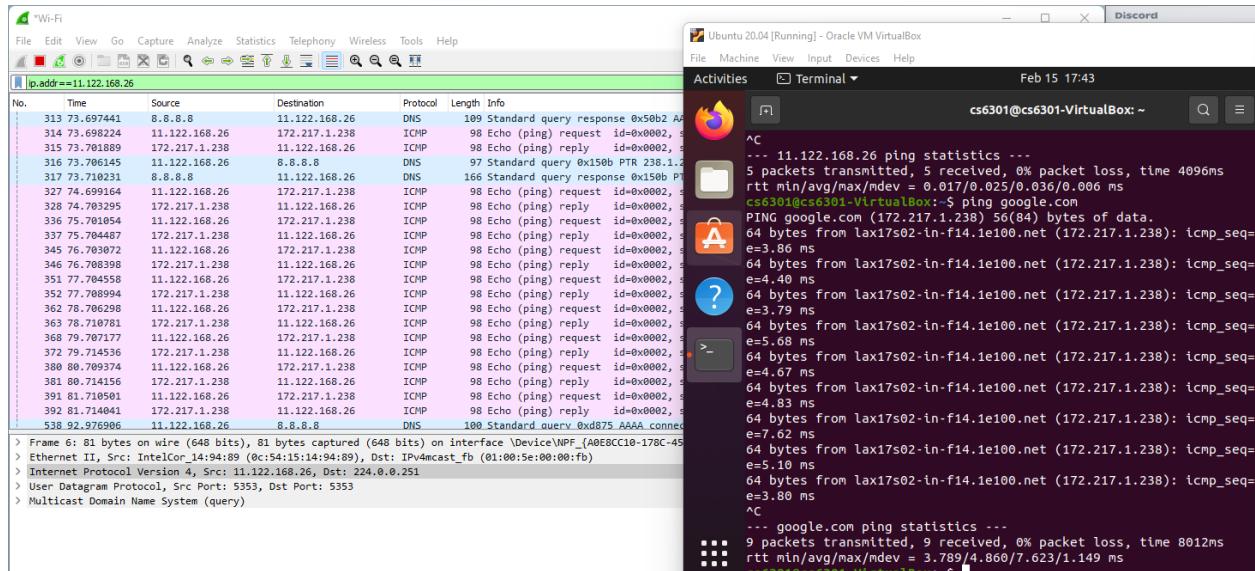
April 8 - Continue research and experiments, set up and train classifier

May 5 - Final Report and Presentation

Final Progress

We began by examining and analyzing current existing models that have collected data in order to overcome and investigate this attack. We start our research through studying *Website Fingerprinting using Traffic Analysis Attack* (Kowsalya), in which the author performs experiments on five most frequently visited sites and surveys website fingerprinting through three phases - data collection, feature extraction, and training and testing. We also focused on WF-GAN: Fighting Back Against Website Fingerprinting Attack Using Adversarial Learning (Hou and Co.) to aid with our modeling and simulating.

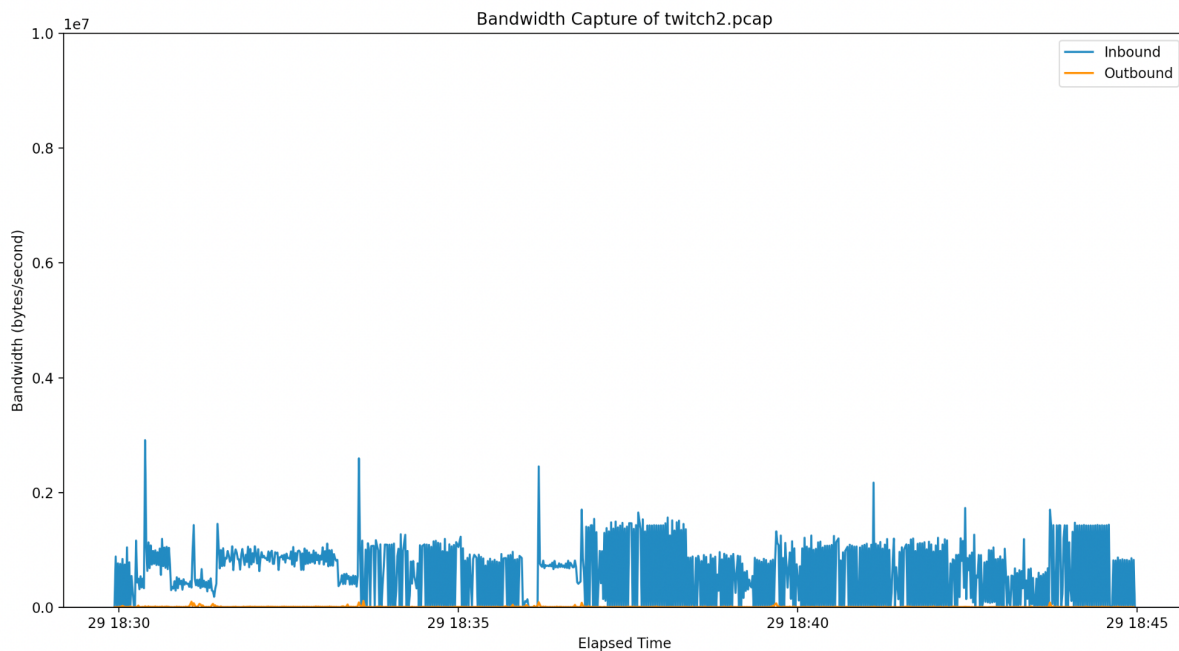
In the past few weeks, our team has set up an environment utilizing Wireshark and conducted initial experiments. Our testing environment consists of setting up a Linux virtual machine to visit websites, as well as setting up Wireshark outside of the virtual machine to read web traffic from the virtual machine.



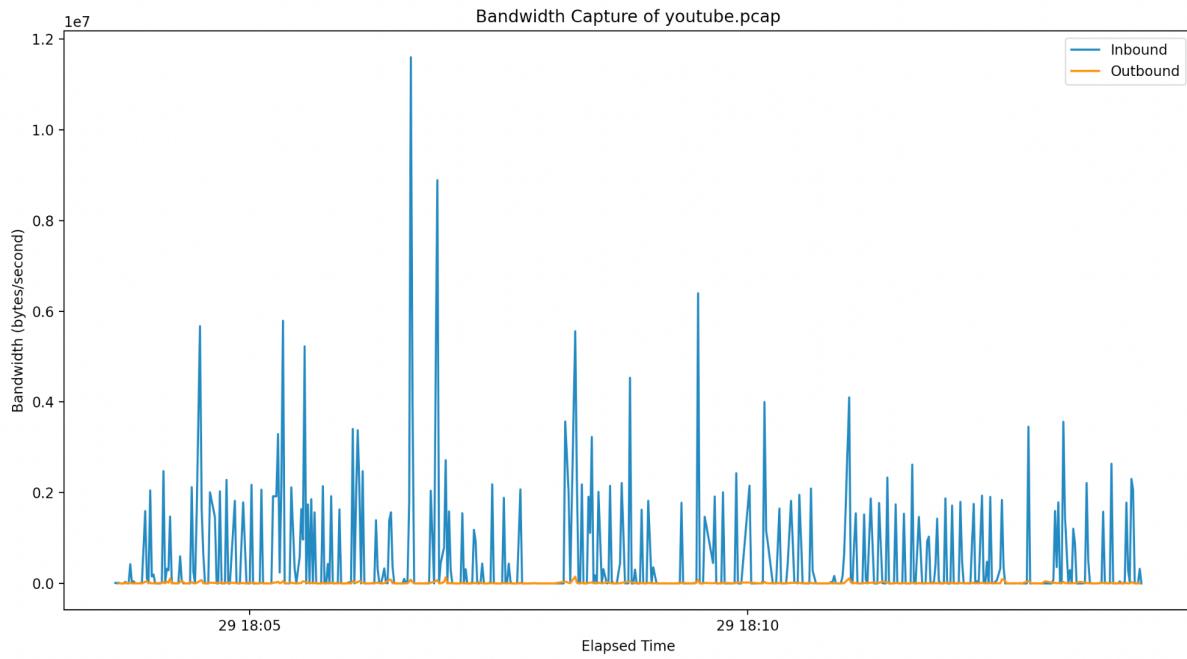
Experimentation

The experiments we have done so far involved collecting 10 minute samples of generic use on websites including Twitch, Reddit, Youtube, and Twitter, as well as the flow metrics (MB/sec and inbound/outbound) for these websites.

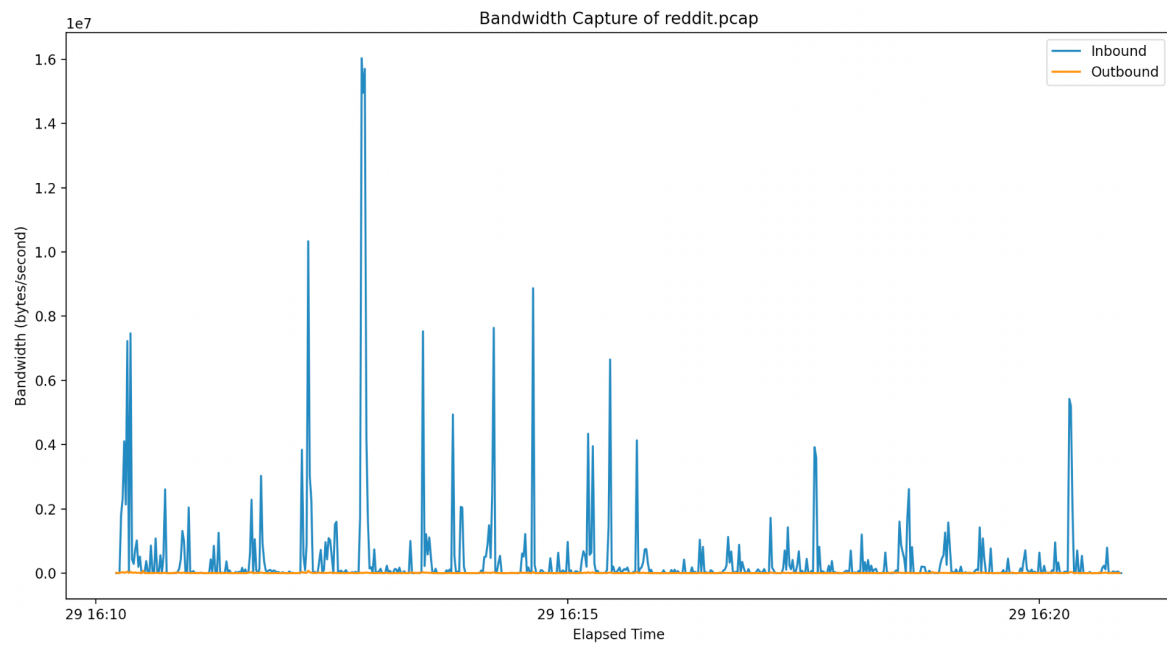
Web Traffic on Twitch



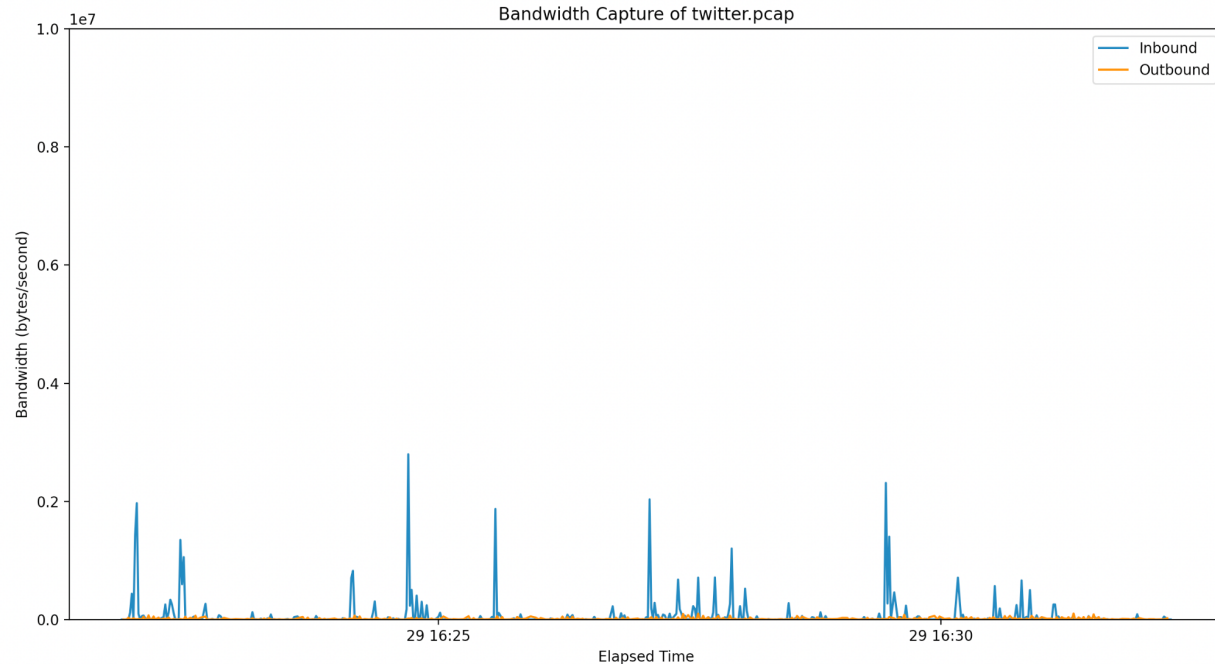
Web Traffic on YouTube



Web Traffic on Reddit



Web Traffic on Twitter



Training the Deep Learning Web Fingerprinting Attack

After capturing and collecting web traffic sessions in PCAP files, the PCAPs will be converted to Pandas DataFrames. Pandas DataFrames are a two dimensional tabular data structure in Python with labels for rows and columns of data. The sizes of the data packets will be aggregated by inbound/outbound inputs and timestamps, the latter of which will be aggregated by the sample rate (i.e. per second). We will repeatedly sample random one-minute segments from the PCAP sessions, with the inputs being inbound 60 sampled MB/second and outbound 60 sampled MB/second, and the label being the website captured by the PCAP. All of this data will then be used to train a model in a Tensorflow neural network over several epochs. After completing the training, the neural network will be tested with PCAP files containing newly captured sessions. Based on the network's results, the model may be adjusted and retested with the aim to improve the model's performance and accuracy.

Weka Explorer

We used Weka to process other classifiers such as Naive Bayes, J48 tree, and experimented with much more as well. Weka Explorer provided a UI that allowed us to preprocess our data, select the classifier, and examine the results after training it with the specified classifier. After parsing the data accurately, we processed 10 samples of each website and selected the specific classifier to train our data set. In the result models below, we can see training performed with the Naive Bayes classification, which produced a result of around 82% of correctly classified sites. The Naive Bayes algorithm is an intuitive method that uses conditional probabilities of each attribute belonging to each class to make a prediction. It uses the Bayes' Theorem to estimate. As you can see in

the confusion matrix, we had one column which was the name of the sites used, so that column of b to b within the matrix is represented as 1. For reddit, 9/10 was classified correctly, with 8/10 for twitch, 9/10 for twitter, and 7/10 for youtube. The author had additionally tested several other classifiers, including tree classifiers - which we tested as well - specifically the J48 tree.

There were lots of other classifiers to select from, however we specifically chose J48. The J48 Classifier is helpful in examining data categorically and continuously, so we decided to go with that classifier. Here, our accuracy grew to 90%, as you can see in the confusion matrix, nearly all sites were guessed accurately - as 10/10 for reddit, 10/10 for twitch, 9/10 for twitter, and 8/10 for youtube. This is the tree produced from the classifier - where you can see how the model split the data to classify the sites.

We were also able to visualize a summary of the results (including the ROC area, accuracy scores, confusion matrix, and tree).

Below are some of our testing results with our data collected.

Naive Bayes

```
Classifier output

Time taken to test model on training data: 0 seconds

=== Summary ===

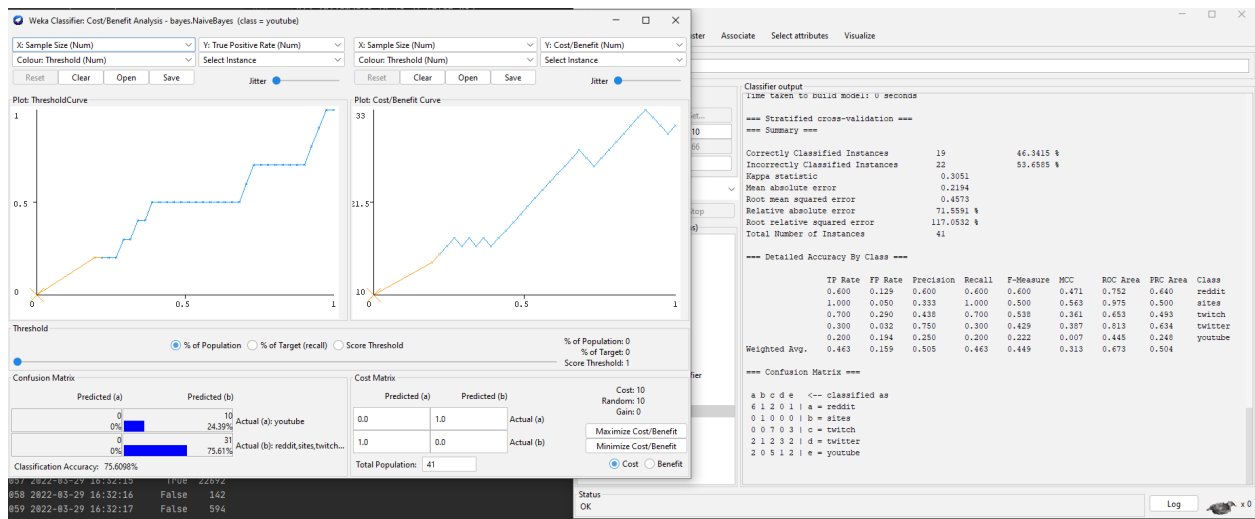
Correctly Classified Instances      34           82.9268 %
Incorrectly Classified Instances    7           17.0732 %
Kappa statistic                    0.7789
Mean absolute error                 0.0753
Root mean squared error            0.2644
Relative absolute error            24.5968 %
Root relative squared error        67.7375 %
Total Number of Instances          41

=== Detailed Accuracy By Class ===

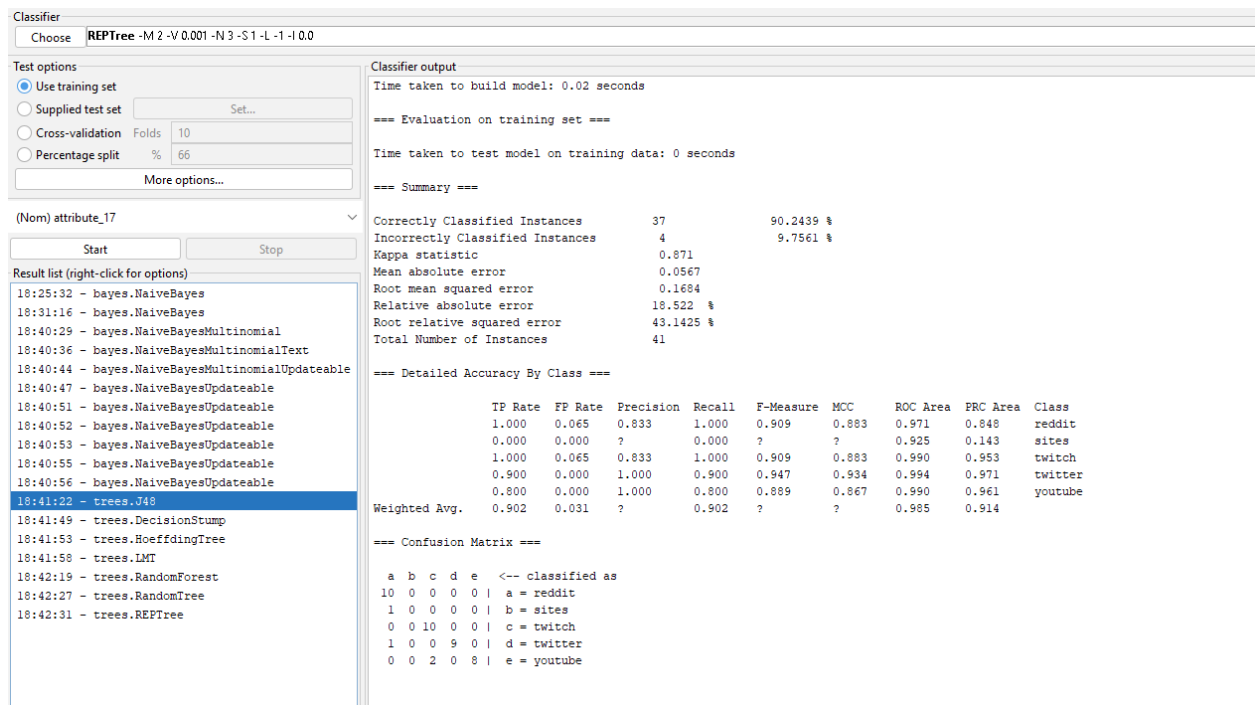
      TP Rate  FP Rate  Precision  Recall   F-Measure  MCC      ROC Area  PRC Area  Class
      0.900   0.129   0.692    0.900   0.783     0.711   0.948    0.896    reddit
      1.000   0.050   0.333    1.000   0.500     0.563   1.000    1.000    sites
      0.800   0.000   1.000    0.800   0.889     0.867   0.994    0.983    twitch
      0.900   0.032   0.900    0.900   0.900     0.868   0.981    0.935    twitter
      0.700   0.000   1.000    0.700   0.824     0.799   0.916    0.883    youtube
Weighted Avg.   0.829   0.041   0.884    0.829   0.840     0.805   0.961    0.926

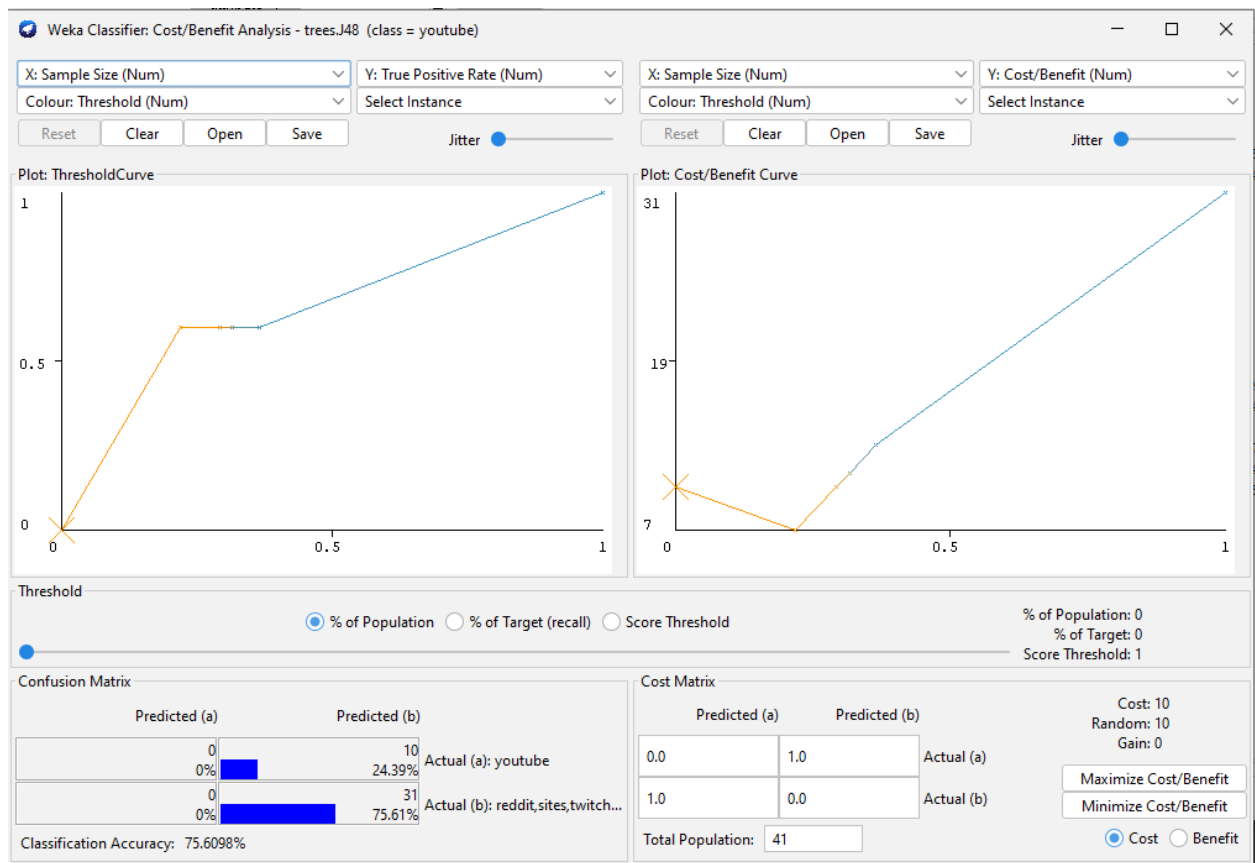
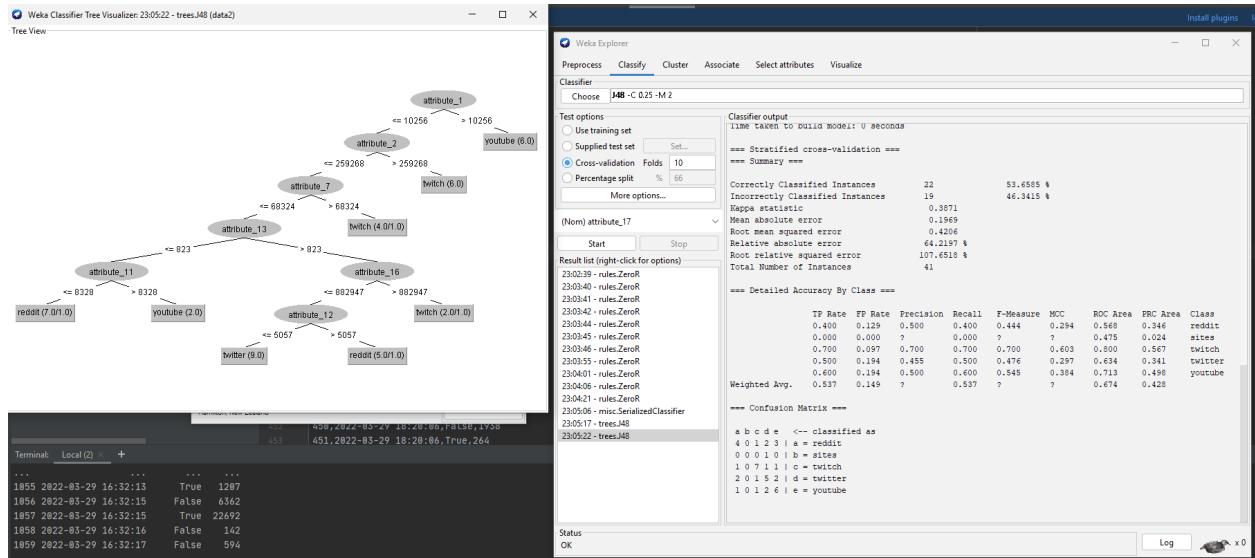
=== Confusion Matrix ===

a b c d e  <-- classified as
9 1 0 0 0 | a = reddit
0 1 0 0 0 | b = sites
2 0 8 0 0 | c = twitch
0 1 0 9 0 | d = twitter
2 0 0 1 7 | e = youtube
```



J48 Tree





Challenges

Some challenges we faced included a steep learning curve from the start of the project, which we combated through thorough research and exploration provided by our sponsor and the internet.

We also faced several challenges figuring out how to successfully get our classifier to accurately classify websites based on web traffic data. These included figuring out the Keras Classification on Google Colab, trying to reason the accuracies of other classifiers by viewing their performance, and having to deal with digesting large amounts of PCAP data for our classifier to use.

Conclusion

We initially started to familiarize ourselves with web traffic fingerprinting by researching various ways to perform traffic fingerprinting attacks and various models for defending against traffic fingerprinting. The team decided to perform experiments that replicate and build upon traffic fingerprinting attacks from the research paper “Website Fingerprinting using Traffic Analysis Attacks” and the WF-GAN model described in the paper “WF-GAN: Fighting Back Against Website Fingerprinting Attack Using Adversarial Learning.”

We decided to use Wireshark together with an Ubuntu virtual machine to collect web traffic data from visiting five different websites via the virtual machine, storing the traffic data as PCAP files. We then randomly sampled our PCAP files into a Pandas dataframe. We then set up a deep learning classifier, based on the Tensorflow neural network model, in Google Colab and trained the classifier to recognize real time traffic using the Pandas dataframes. We also used Weka to process other classifiers such as Naive Bayes and J48 tree to see how well they recognized real time traffic.

Our deep learning classifier had an 81.8% accuracy, Naive Bayes had an 82.9268% accuracy, and the J48 tree had a 90.2439% accuracy. Website fingerprinting attacks are always possible. With the method used, we can obtain up to an accuracy of 90.2439% using the data from bandwidth and average packet transferred per second.

Future Plans

We completed training a classifier to recognize usage patterns, like a website fingerprinting attack. Our next steps to improve our classifier would be to train our classifier to handle a user interleaving websites and traffic patterns differences between browsers. We would then implement a real time tool to automatically classify live packets. After making these improvements to the classifier, our next steps would be to build an adversarial network and test it to see if it can fool the classifier.

Literature Review

What Are You Searching For? A Remote Keylogging Attack on Search Engine Autocomplete [6]

In most technologies used today, an autocomplete feature is constantly used to efficiently produce user requests and queries. This gives vulnerability to privacy, as it also uses network traffic from the client when changing the autocomplete - which can be leaked for attack. This keylogging attack on the autocomplete engine can be induced by three sources: timing of keystrokes predicted, percent encoded

“space” characters used, and the Huffman code in HTTP2 header compression. This attack engine developed in the paper called KREEP (Keystroke Recognition and Entropy Elimination Program) succeeds even while the traffic is encrypted. In Monaco’s paper, he examines the pattern of autocomplete packet sizes from the search engine, a side channel attack using Huffman code, a neural network that predicts words from keystroke timings, and leveraging the English language to examine entropy. The threat model assumes a remote adversary that has access to a dataset of the English language, and has five stages of pipeline architecture - keystroke detection, tokenization (binary classification by Space characters), dictionary pruning (filtering of words), word identification (probability per word), and beam search (predicted words). KREEP is first tested under ideal situations, but then gradually tested under increasing levels of noise. As a result, the attack succeeds on more than half of the characters in queries on average. It is beneficial to identify the vulnerability and the attack methods utilized in this keylogging attack.

KeyDrown: Eliminating Software-Based Keystroke Timing Side-Channel Attacks [2]

Most computers and software are vulnerable to information leaks through side channels, which are often accomplished through keystroke timing attacks. Although there exist several mechanisms for defending against keystroke timing attacks, they do not offer protection against all keystroke attacks because they do not address most of the layers involved in keystroke processing. Most of these defense mechanisms only focus on system interfaces. This paper proposes a solution to this problem by defining a new defense mechanism against keystroke timing attacks called KeyDrown. KeyDrown is built to meet three established requirements for countering keystroke timing attacks: random classifiers must have an advantage over based on a single-trace side channel attack, classifiers must use an extremely high amount of side-channel traces to correctly identify keystrokes with full accuracy, and the implementation must be leak-free. KeyDrown covers the entire software stack and combats keystroke attacks by injecting many fake keystrokes to trick the attacker.

Fingerprinting Encrypted Voice Traffic on Smart Speakers with Deep Learning [3]

With the popularization of network technology, a large number of smart home appliances have inevitably entered everyone’s life. At the same time, people’s concerns about the leakage of personal information and privacy have also hindered the development of smart home appliances to a certain extent. Among all smart home appliances, the one that is most worried about privacy leakage is the smart speaker.

This paper discussed the degree of security of smart speakers under an encrypted traffic analysis attack. In conclusion, this attack can eavesdrop bidirectional encrypted voice traffic of a smart speaker and corresponding cloud servers to leverage side-channel information, which are size, direction, and order of encrypted packets, to speculate users' voice information without decryption. Leaked voice information can not only expose the user's privacy but also the information can be used for illegal and criminal activities and cause immeasurable losses. The author of this article uses various methods to improve the accuracy of hacking attacks to better understand the principle of privacy leakage and prevent such hacking attacks. The author performed the hacking attacks based on CNN, LSTM, and SAE respectively and it turns out that CNN attains the highest accuracy among them. Based on the proof-of-concept attacks, the author's team designed a proof-of-concept defense mechanism which obfuscates traffic and mitigates privacy leakage against voice command fingerprinting. This defense mechanism can reduce attack accuracy to 1.23% and if attacker adapts attack accuracy can still be reduced to 32.18%. This research shows that putting safety measures on incoming traffic only is not enough and the defense mechanism proposed by the author's team can greatly reduce the risk of privacy leakage.

Website Fingerprinting using Traffic Analysis Attacks [4]

While encrypting network traffic has been the norm for several years, encryption is incapable of obscuring the coarser grained features that can be intercepted with any MitM attack. Some example features include packet length, the interval between packets, upstream bandwidth, and downstream bandwidth. These features aren't obscured by encryption because they are inherent to the entire networking process; packets must be sent upstream/downstream through a vulnerable though obscured channel. The author collected their data by accessing 5 popular websites and utilizing a packet sniffer in between their machine and the access point. With the data from the websites collected, they trained several different classifiers and were able to accurately predict which website was being used based on the observed features. These results proved that packet sniffing can always be carried out even with packet encryption.

WF-GAN: Fighting Back Against Website Fingerprinting Attack Using Adversarial Learning [5]

Web Fingerprinting is the technique of sniffing encrypted packets and classifying which website a user is browsing. This attack is becoming very difficult to overcome, especially with the advent of Deep Learning classifiers that are capable of deciding with the raw flow-based features very accurately. These improved techniques have stumped older traffic modifiers such as Walkie-Talkie. Thus, the authors proposed a Generative Adversarial Network to modify the traffic of a website. First, the data would be

converted into input/output burst sequences and fed to the network, split into the original data and fake, generated data. This data is then tested by the discriminator to see if the data is plausible enough to be taken as a normal packet. They pitted this network against the state of the art Deep Learning WF attack and allowed it to be trained until its loss had been minimized. By epoch 300, the targeted defenses were able to perform much better than Walkie-Talkie. They further tuned the network by allowing more overhead in terms of packet size and frequency, so the network was able to fool the Deep Learning classifier. This method successfully obscured the data against the Web Fingerprinting attack.

Speaker recognition from encrypted VoIP communications [7]

More VoIP applications are using encryption for speech communication transmissions, and tracing the identities of the speakers in these channels becomes a lot harder to achieve due to the speech being encrypted. This is a major issue for forensic investigations that need to identify perpetrators in a conversation carried out in a VoIP application. This paper proposes multiple different techniques for identifying and verifying speakers in encrypted VoIP conversations and provides details of the experiments conducted to test these techniques. Among the different techniques, speaker-specific information from information can be extracted using variable packet length information if variable bit speech encoding is used. The techniques' accuracy in identification and verification in the experiments are much higher than the accuracy provided by random guessing, which makes these techniques potentially suitable for supplementing forensic evidence.

References

- [1] Meng, Yitong, and Jinlong Fei. "Hidden Service Website Response Fingerprinting Attacks Based on Response Time Feature." *Security and Communication Networks*, Hindawi, 1 Dec. 2020, <https://www.hindawi.com/journals/scn/2020/8850472/>.
- [2] Schwarz, Michael, et al. "KeyDrown: Eliminating Software-Based Keystroke Timing Side-Channel Attacks." *Proceedings 2018 Network and Distributed System Security Symposium*, 2018, <https://doi.org/10.14722/ndss.2018.23027>.
- [3] Wang, Chenggang, et al. "Fingerprinting Encrypted Voice Traffic on Smart Speakers with Deep Learning." *Proceedings of the 13th ACM Conference on Security and Privacy in Wireless and Mobile Networks*, 2020, <https://doi.org/10.1145/3395351.3399357>.
- [4] Kowsalya, Salini Selvaraj. "Website Fingerprinting using Traffic Analysis Attacks." 2017, https://pages.cs.wisc.edu/~salinisk/642/website_fingerprinting_paper.pdf
- [5] C. Hou, G. Gou, J. Shi, P. Fu and G. Xiong, "WF-GAN: Fighting Back Against Website Fingerprinting Attack Using Adversarial Learning," 2020 IEEE Symposium on Computers and Communications (ISCC), 2020, pp. 1-7, doi: 10.1109/ISCC50000.2020.9219593.
- [6] *What Are You Searching for? A Remote Keylogging ... - Usenix.*
<https://www.usenix.org/system/files/sec19-monaco.pdf>.
- [7] L.A. Khan, M.S. Baig, Amr M. Youssef, Speaker recognition from encrypted VoIP communications, *Digital Investigation*, Volume 7, Issues 1–2, 2010, Pages 65-73, ISSN 1742-2876, <https://doi.org/10.1016/j.diin.2009.10.001>.

Investigator Biographies

Ruiyu Mao

Ruiyu Mao is a first year student in the master program of computer science with a concentration in Information Assurance. He was an aerospace engineer before . Although he has limited computer security related experience as an aerospace engineer, he received fundamental computer science education at the undergraduate level and is currently taking courses related to information security and cybersecurity at UTD. He is experienced in java, Python, MATLAB and able to use C/C++.

Joseph Norman

Joseph is a student at UTD studying Computer Science with an emphasis on CyberSecurity. He has been present in many Cybersecurity courses and has been involved in writing networking programs in his other related courses. He is able to use Python, Java, C/C++, Lisp, and Prolog. While at his internship, he participated on a team of developers working to create applications and learned a lot.

Tram Pham

Tram Pham is a graduate student studying cyber security at The University of Texas at Dallas. She is a recipient of the CyberCorps Scholarship for Service (SFS). She was interned with Kessel Run over the summer 2021. During her internship, she worked in developing an application in django that ensures to have enough security configurations requirements and also to integrate the application with the company's 2 oauth authentication. She has completed related courseworks in Network Security, Information Security, Machine learning and Web programming. She is experienced in Python, Java, Javascript, C/C++, MIPS and SQL.

Marlon Schumacher

Marlon is currently a senior completing his Bachelor's in Computer Science and is a Fast-Track student in the Master's in Computer Science degree program with a focus in cybersecurity at the University of Texas at Dallas. He has completed courses in computer networks and web security, and is currently taking courses in information security and systems security. During his time as a member of the Computer Security Group on campus, Marlon gained an understanding of basic cybersecurity concepts and practiced completing capture-the-flag challenges. He has experience with Java, C++, C, C#, SQL, JavaScript, PHP, and Python from coursework at UT Dallas and his summer 2021 internship at TeamCalendar.ai.

Alinta Wang

Alinta has recently graduated with a Bachelor's of Science in Computer Science this past Spring at the University of Texas at Dallas, and is looking to pursue her Master's degree through Fast Track with a focus in Cybersecurity at UTD as well. She has taken courses in Information Security, Network Security, and Big Data Security and Privacy. She is familiar with coding in languages such as Java, SQL, C, and shell. She has interned at Intel Security and recently finished an internship at Fidelity Investments as well, and hopes to advance and utilize her knowledge in future projects.